Simulated Autonomous Maze Navigation by a Robotic System

YINA CAI

Introduction

- Simulation of autonomous maze navigation using a physical mobile robot, TurtleBot3, equipped with 360 Laser Distance Sensor LDS-02. I utilized the Robot Operating System (ROS) for robot control and Gazebo for creating a simulated maze environment. The primary goal is to simulate and experiment a virtual and physical mobile robot to independently explore mazes, build maps of its surroundings in real time using the Simultaneous Localization and Mapping (SLAM) algorithm, and make smart decisions to avoid obstacles.

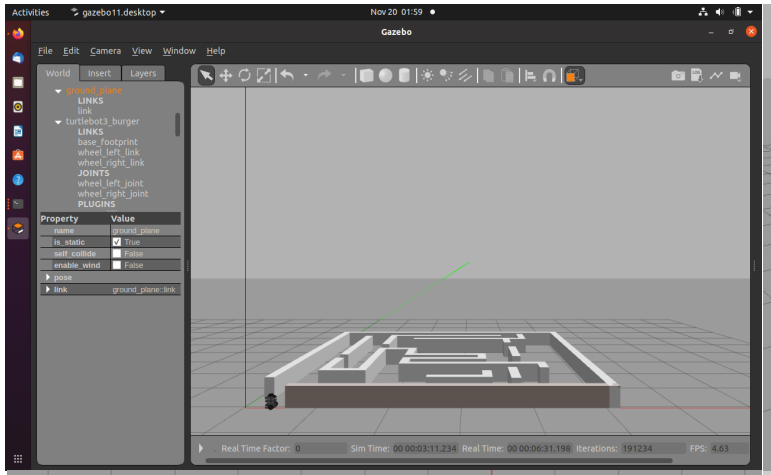
Problem Description

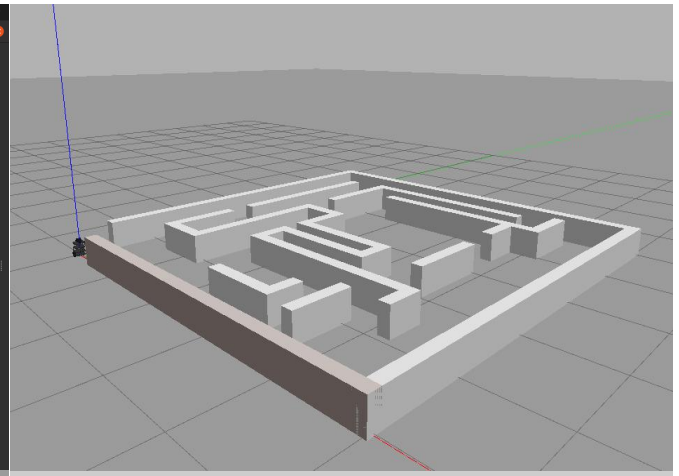
- In numerous real-life scenarios, ranging from search and rescue mission in hazardous or unstable environments to optimize warehouse operations by efficiently navigating through storage spaces to retrieve and deliver goods, there is a growing demand for robots intelligently navigate complex and dynamic spaces while avoiding obstacles.
- This project seeks to implement the SLAM algorithm using the Turtlebot3 and the Rao-Blackwellized particle filter. Implementing the simulation on ROS and real physical environment, I chose the GMapping algorithm for several reasons.

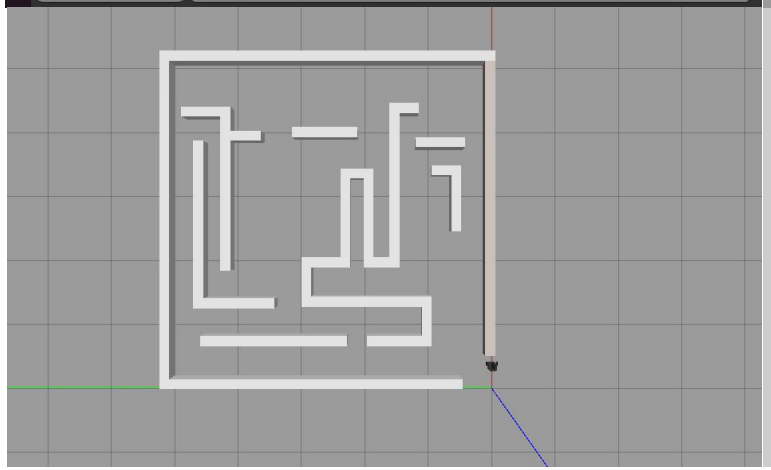
Distinct advantages of GMapping

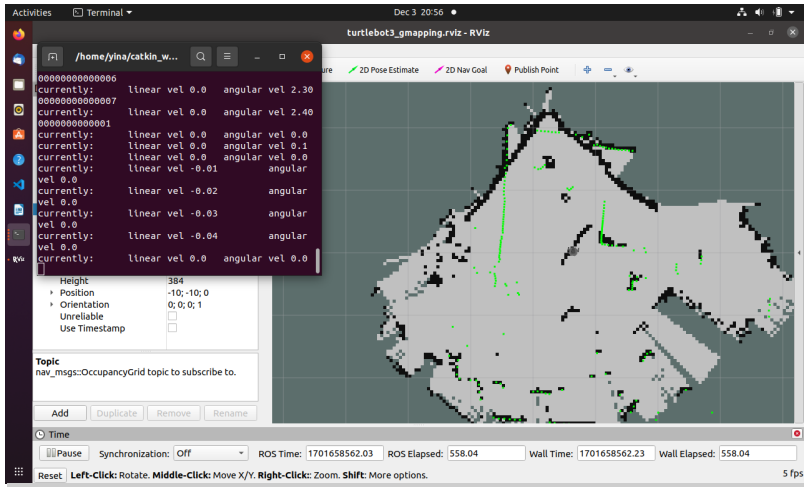
- It employs an enhanced proposal distribution, integrating high-precision lidar data and adaptive resampling, resulting in a detailed two-dimensional grid map.
- GMapping demonstrates fault tolerance for data correlation, providing a more precise posterior probability estimate of the map.
- As a ROS node, GMapping focuses on laser-based SLAM, requiring raw laser scan data, thus aligning well with TurtleBot3's 360 Laser Distance Sensor LDS-02.
- This approach utilizes a reduced number of particles for robot state estimation, effectively addressing the issue of particle degeneration. By minimizing variance in particle weight, GMapping improves upon traditional SLAM algorithms, particularly when facing significant motion noise challenges.

Results


Activities
gazebo11.desktop
Nov 20 01:59
Gazebo
File Edit Camera View Window Help
World Insert Layers
ground_plane
LINKS
link
turtlebot3_burger
LINKS
base_footprint
wheel_left_link
wheel_right_link
JOINTS
wheel_left_joint
wheel_right_joint
PLUGINS
Property Value
name ground_plane
is_static True
self_collide False
enable_wind False
pose
link ground_plane::link
Real Time Factor: 0
Sim Time: 00 00:03:11.234
Real Time: 00 00:06:31.198
Iterations: 191234
FPS: 4.63






Activities
Terminal
Dec 3 20:56
turtlebot3_gmapping.rviz - RViz
/home/yina/catkin_w...
0000000000006
currently: linear vel 0.0 angular vel 2.30
0000000000007
currently: linear vel 0.0 angular vel 2.40
0000000000001
currently: linear vel 0.0 angular vel 0.0
currently: linear vel 0.0 angular vel 0.1
currently: linear vel 0.0 angular vel 0.0
currently: linear vel -0.01 angular vel 0.0
currently: linear vel -0.02 angular vel 0.0
currently: linear vel -0.03 angular vel 0.0
currently: linear vel -0.04 angular vel 0.0
currently: linear vel 0.0 angular vel 0.0
2D Pose Estimate
2D Nav Goal
Publish Point
Height 384
Position -10; -10; 0
Orientation 0; 0; 0; 1
Unreliable
Use Timestamp
Topic
nav_msgs::OccupancyGrid topic to subscribe to.
Add Duplicate Remove Rename
Time
Pause
Synchronization: Off
ROS Time: 1701658562.03
ROS Elapsed: 558.04
Wall Time: 1701658562.23
Wall Elapsed: 558.04
Reset
Left-Click: Rotate. Middle-Click: Move X/Y. Right-Click:: Zoom. Shift: More options.
5 fps

Findings/Insights

- If an obstacle is not tall enough (below the Laser Sensor), it might go undetected.
- In certain scenarios, Turtlebot3 can get stuck in map corners, even without collisions, indicating scope for improved accuracy of mapping.
- Before completing the previous path, if a new destination is set, the navigation algorithm halts the previous path and recalculates a new plan for the updated destination.
- Each time a new destination is assigned, the navigation system consistently selects the shortest route to reach it. (using Dijkstra's algorithm)
- BFS vs. Dijkstra's Algorithm
  - BFS(unweighted graphs) – $O(V + E)$
  - Dijkstra's Algorithm(weighted graphs) – $O(V + E\ (\log V))$